

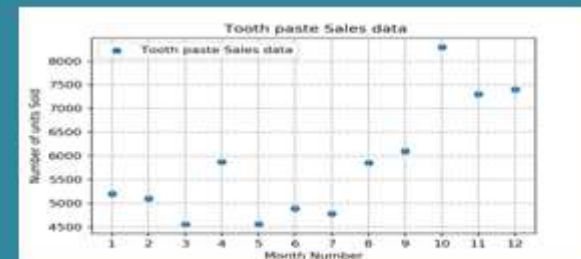
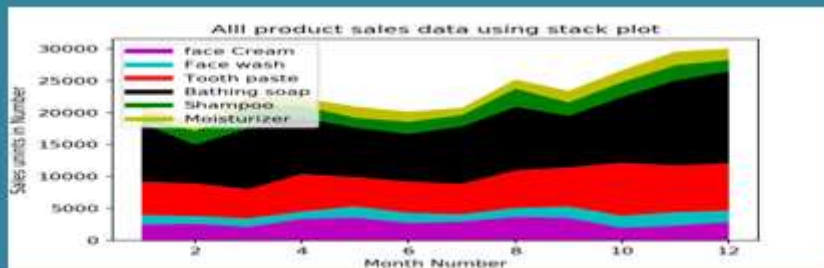
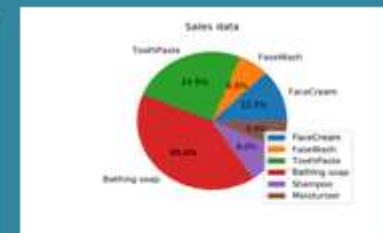
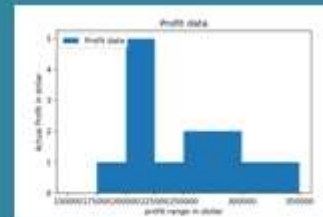
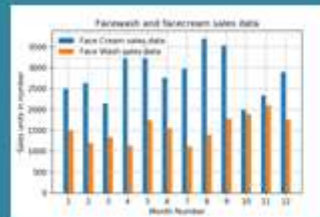
What is Matplotlib?

- Matplotlib is a comprehensive library for creating static, animated, and interactive visualizations in Python. It is an open-source and free to use.
- Plotting of data can be extensively made possible in an interactive way by Matplotlib, which is a plotting library that can be demonstrated in Python scripts.
- Plotting of graphs is a part of data visualization, and this property can be achieved by making use of Matplotlib.



Python Matplotlib Exercise

Practice Data Visualization In, Practice Questions Online, Solution Provided for Each Question



In []: ▶

In [1]: ▶

```
!pip install matplotlib
```

```
Requirement already satisfied: matplotlib in c:\users\admin\anaconda3\lib\site-packages (3.7.1)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\admin\anaconda3\lib\site-packages (from matplotlib) (1.0.5)
Requirement already satisfied: cycler>=0.10 in c:\users\admin\anaconda3\lib\site-packages (from matplotlib) (0.11.0)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\admin\anaconda3\lib\site-packages (from matplotlib) (4.25.0)
Requirement already satisfied: kiwisolver>=1.0.1 in c:\users\admin\anaconda3\lib\site-packages (from matplotlib) (1.4.4)
Requirement already satisfied: numpy>=1.20 in c:\users\admin\anaconda3\lib\site-packages (from matplotlib) (1.24.3)
Requirement already satisfied: packaging>=20.0 in c:\users\admin\anaconda3\lib\site-packages (from matplotlib) (23.0)
Requirement already satisfied: pillow>=6.2.0 in c:\users\admin\anaconda3\lib\site-packages (from matplotlib) (9.4.0)
Requirement already satisfied: pyparsing>=2.3.1 in c:\users\admin\anaconda3\lib\site-packages (from matplotlib) (3.0.9)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\admin\anaconda3\lib\site-packages (from matplotlib) (2.8.2)
Requirement already satisfied: six>=1.5 in c:\users\admin\anaconda3\lib\site-packages (from python-dateutil>=2.7->matplotlib) (1.16.0)
```

In []: ▶

In [4]: ▶

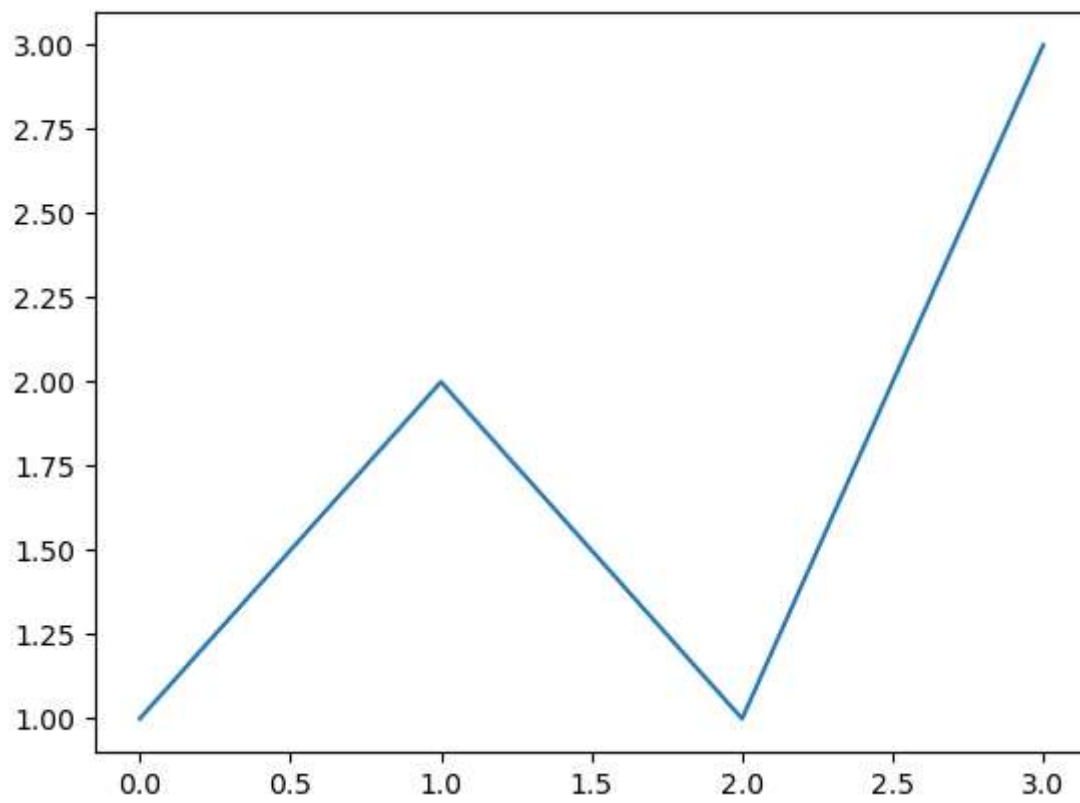
```
import numpy as np
import matplotlib.pyplot as plt
```

In [3]: ▶

```
[1,2,1,3]
```

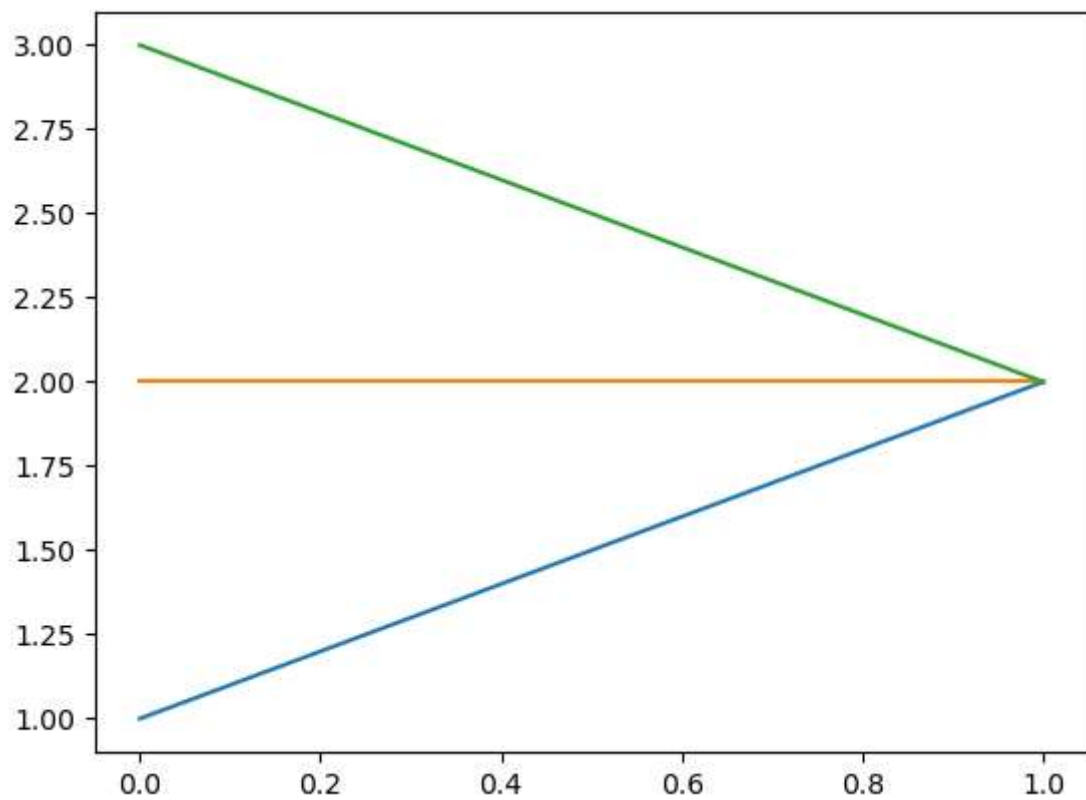
Out[3]: [1, 2, 1, 3]

```
In [5]: ▶ plt.plot([1,2,1,3])  
plt.show()
```

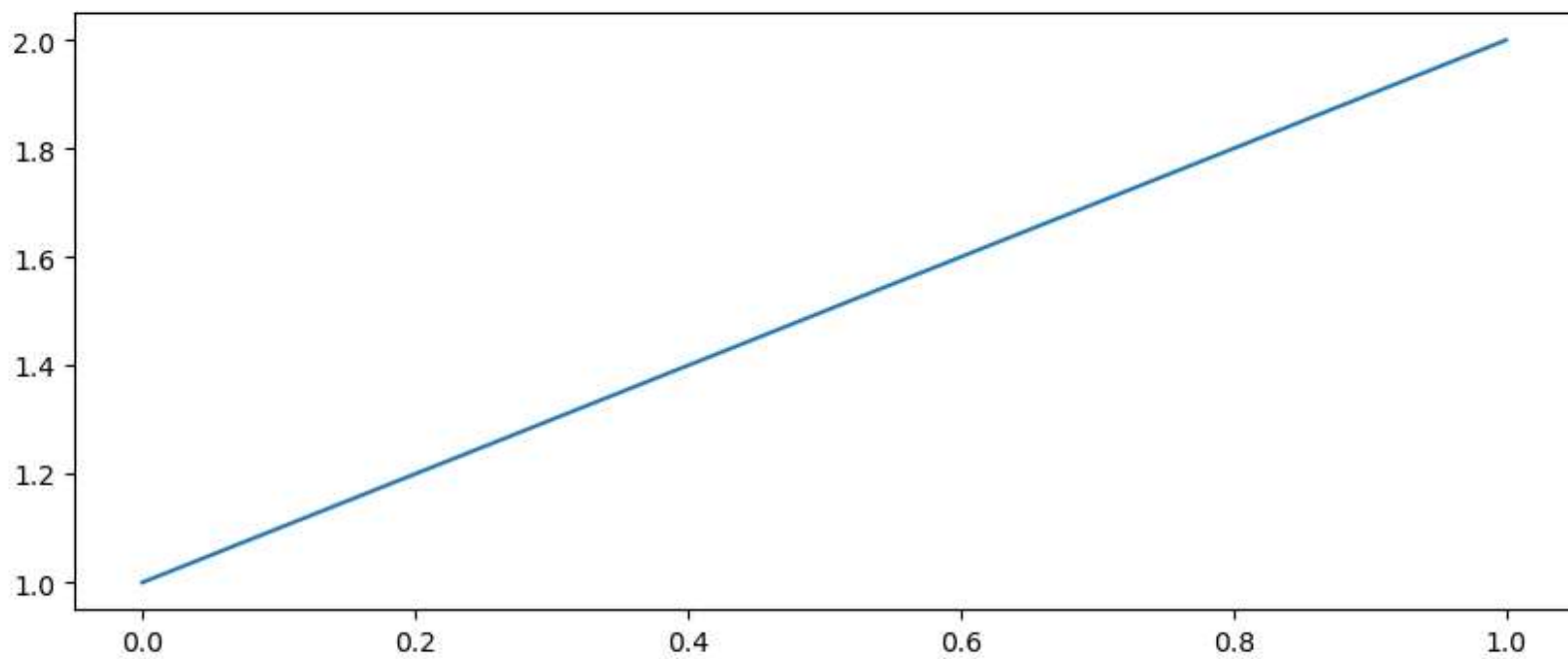


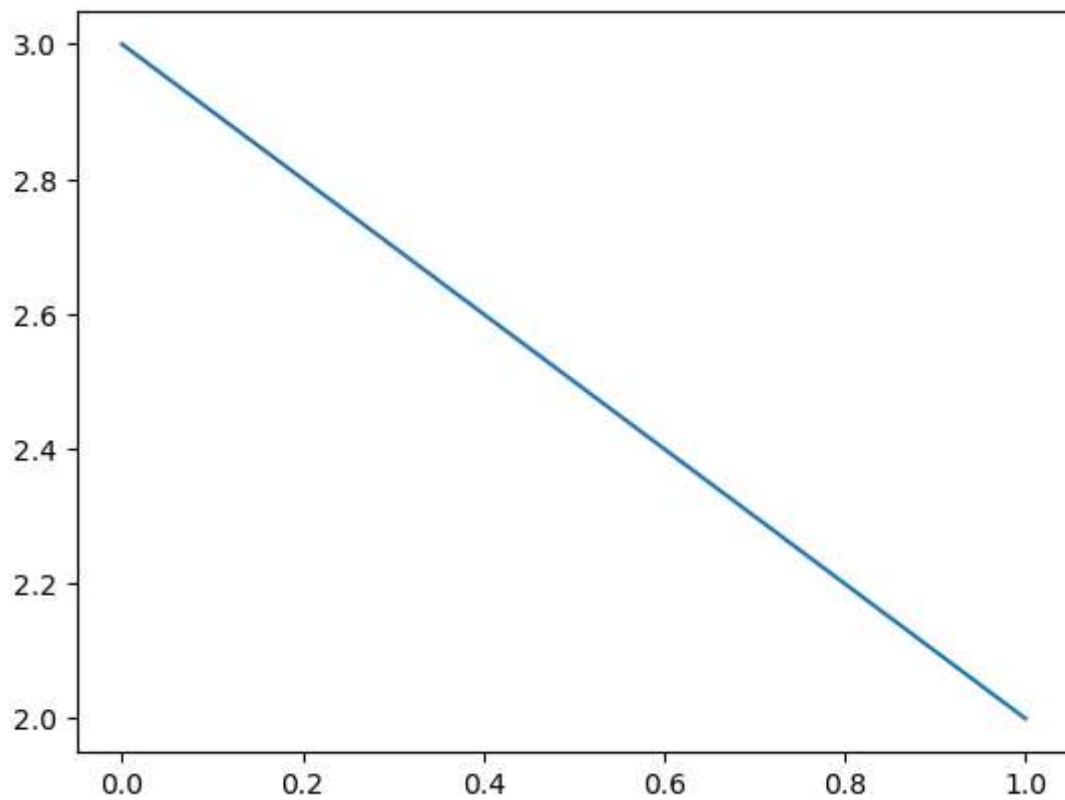
```
In [ ]: ▶
```

```
In [6]: ▶ plt.plot([1,2])  
plt.plot([2,2])  
plt.plot([3,2])  
  
plt.show()
```



```
In [9]: ▶ plt.figure(1, figsize=(10, 4))  
plt.plot([1,2])  
  
plt.figure(2)  
plt.plot([3,2])  
  
plt.show()
```

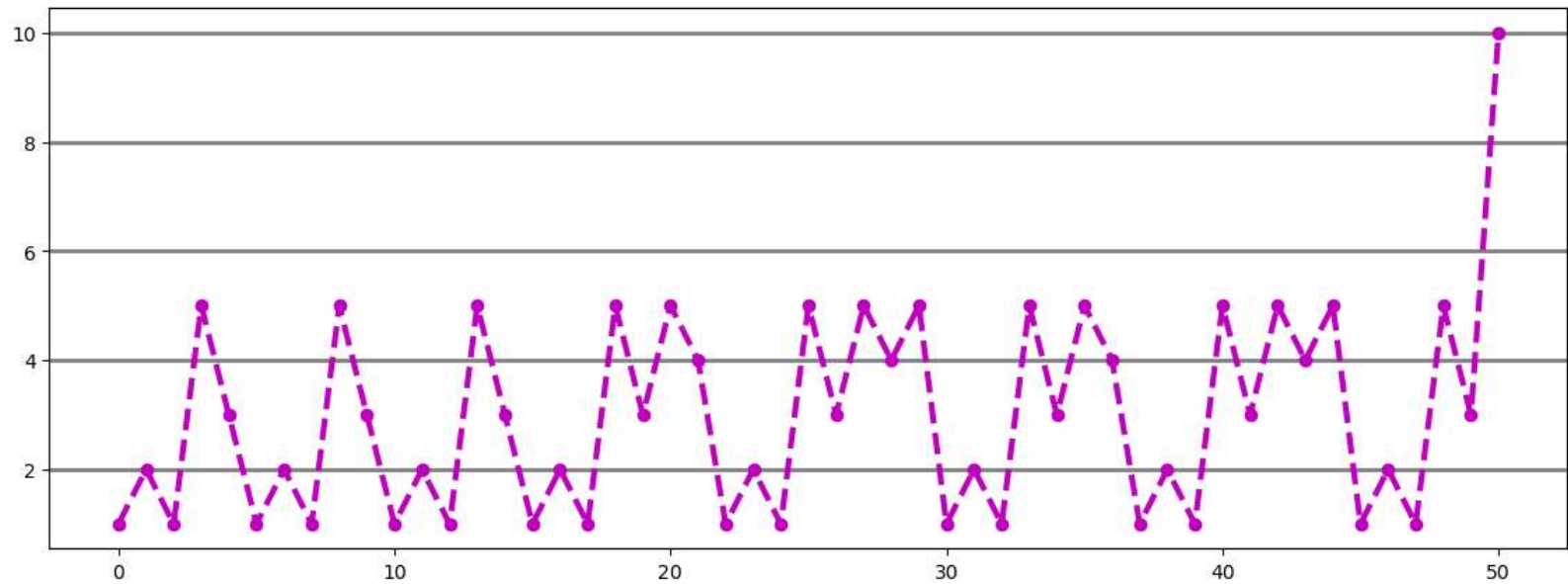




In []: ▶

```
In [50]: ▶ plt.figure(figsize=(14, 5))
plt.plot([1,2,1,5,3,1,2,1,5,3,1,2,1,5,3,1,2,1,5,3,5,4,1,2,1,5,3,5,4,5,
         1,2,1,5,3,5,4,1,2,1,5,3,5,4,5,1,2,1,5,3,10], color='m', marker='o', markersize=6,
         linewidth=3, linestyle='dashed')

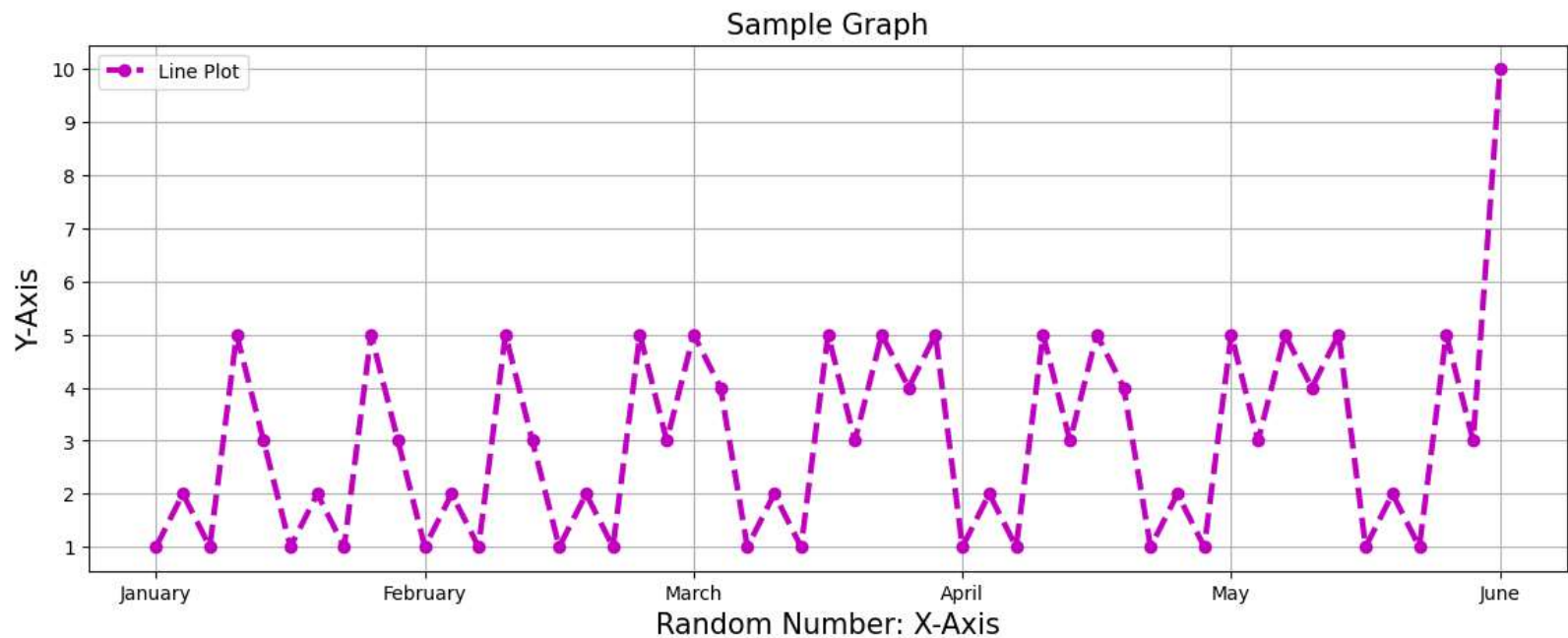
plt.grid(axis='y', color='grey', linewidth=2)
plt.show()
```



```
In [84]: ▶
```

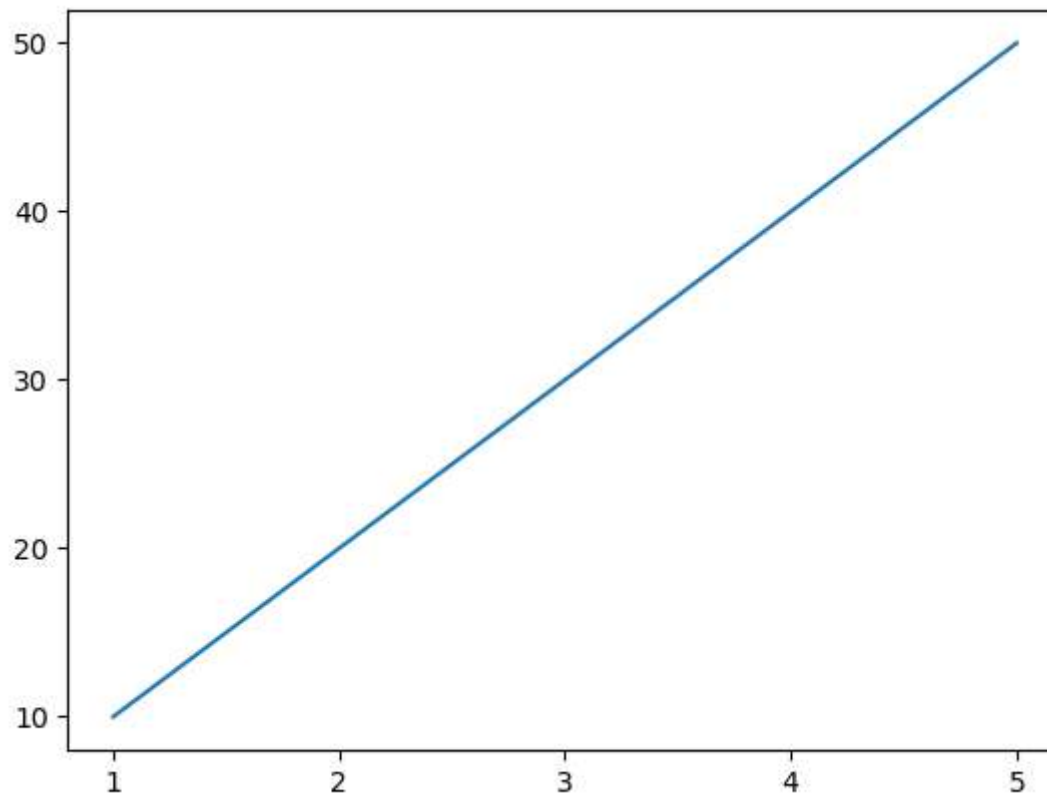
```
In [85]: ▶ plt.figure(figsize=(14, 5))
plt.plot([1,2,1,5,3,1,2,1,5,3,1,2,1,5,3,1,2,1,5,3,5,4,1,2,1,5,3,5,4,5,
          1,2,1,5,3,5,4,1,2,1,5,3,5,4,5,1,2,1,5,3,10], color='m', marker='o', markersize=6,
          linewidth=3, linestyle='dashed', label='Line Plot')

plt.title('Sample Graph', fontsize=15, loc='center')
plt.xlabel('Random Number: X-Axis', fontsize=15,)
plt.ylabel('Y-Axis', fontsize=15)
plt.xticks([0, 10, 20, 30, 40, 50], ['January', 'February', 'March', 'April', 'May', 'June'])
plt.yticks(np.arange(1,11,1))
plt.grid()
plt.legend()
plt.show()
```




```
In [90]: ▶ x = [1,2,3,4,5]
y = [10, 20, 30, 40, 50]

plt.plot(x, y)
plt.xticks(np.arange(1,6,1))
plt.yticks(np.arange(10,51,10))
plt.show()
```



```
In [ ]: ▶
```

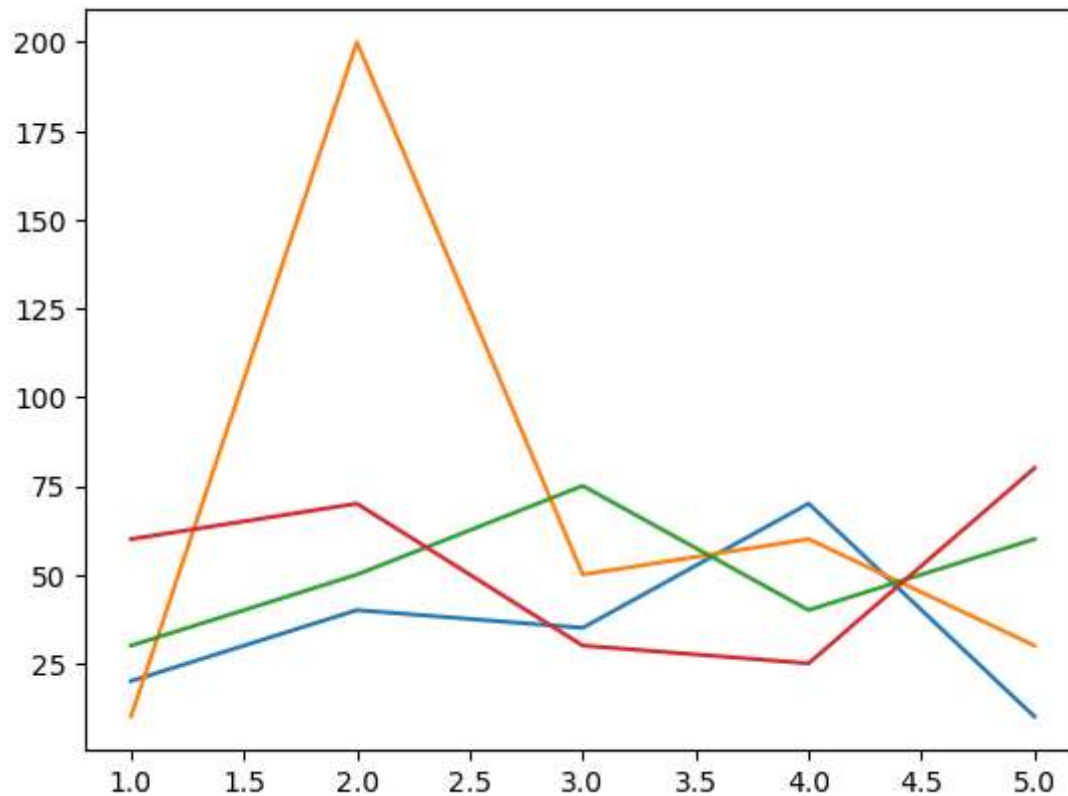
```
In [ ]: ▶
```

Line Plot

```
In [93]: x = [1, 2, 3, 4, 5]

y1 = [20, 40, 35, 70, 10]
y2 = [10, 200, 50, 60, 30]
y3 = [30, 50, 75, 40, 60]
y4 = [60, 70, 30, 25, 80]

plt.plot(x, y1)
plt.plot(x, y2)
plt.plot(x, y3)
plt.plot(x, y4)
plt.show()
```



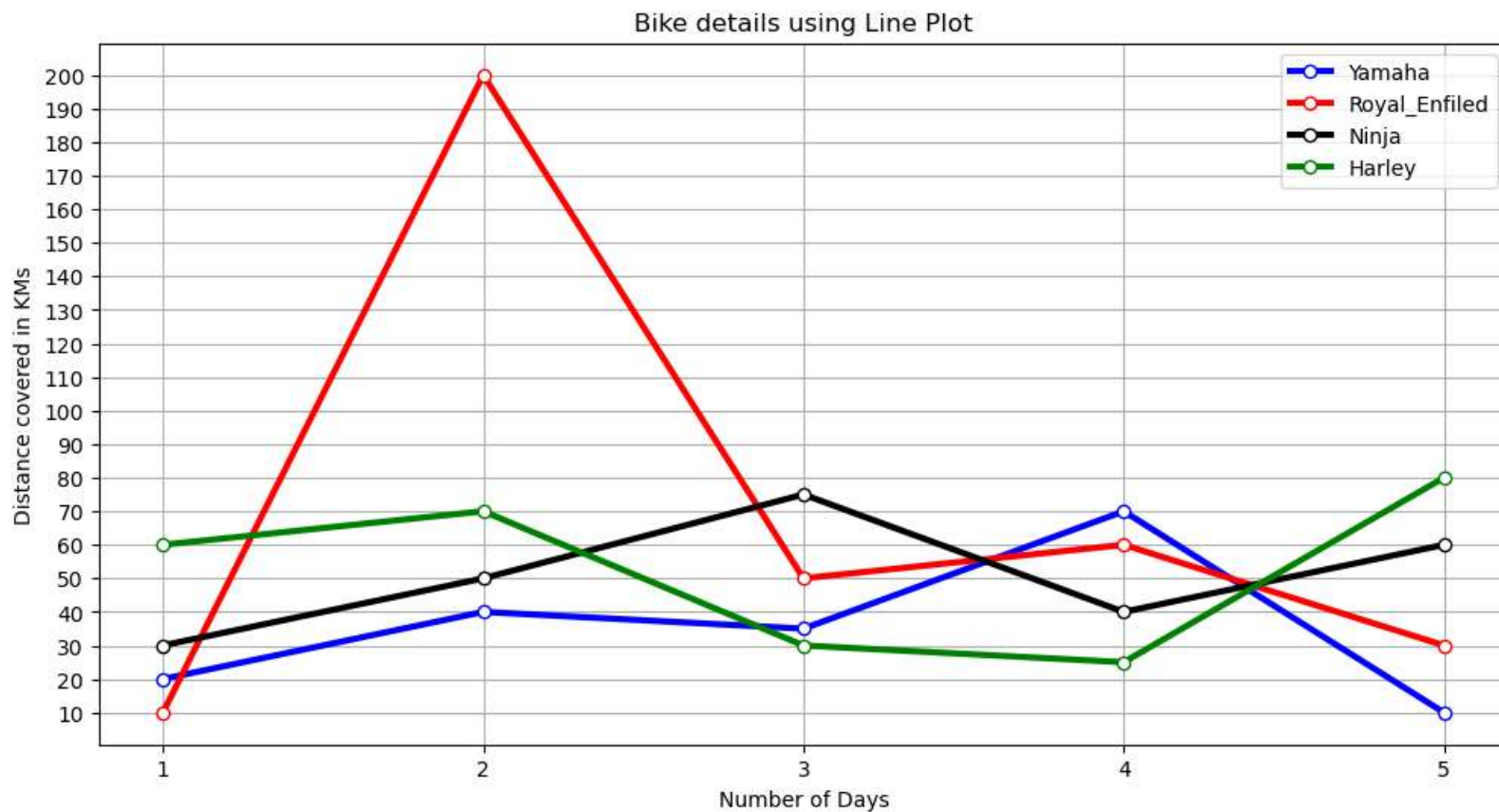
```
In [ ]: ▶ # marker: marker style string, `~.path.Path` or `~.markers.MarkerStyle`  
#         markededgecolor or mec: color  
#         markededgewidth or mew: float  
#         markerfacecolor or mfc: color  
#         markerfacecoloralt or mfcalt: color  
#         markersize or ms: float
```

```
In [113]: ▶ x = [1, 2, 3, 4, 5]

y1 = [20, 40, 35, 70, 10]
y2 = [10, 200, 50, 60, 30]
y3 = [30, 50, 75, 40, 60]
y4 = [60, 70, 30, 25, 80]

plt.figure(figsize=(12, 6))
plt.plot(x, y1, color='b', marker='o', label='Yamaha', lw=3, markerfacecolor='white')
plt.plot(x, y2, color='r', marker='o', label='Royal_Enfield', lw=3, mfc='w')
plt.plot(x, y3, color='k', marker='o', label='Ninja', lw=3, mfc='w')
plt.plot(x, y4, color='g', marker='o', label='Harley', lw=3, mfc='w')

plt.title('Bike details using Line Plot')
plt.xlabel('Number of Days')
plt.ylabel('Distance covered in KMs')
plt.xticks(np.arange(1,6,1))
plt.yticks(np.arange(10,201,10))
plt.legend()
plt.grid()
plt.show()
```



In []: ▶

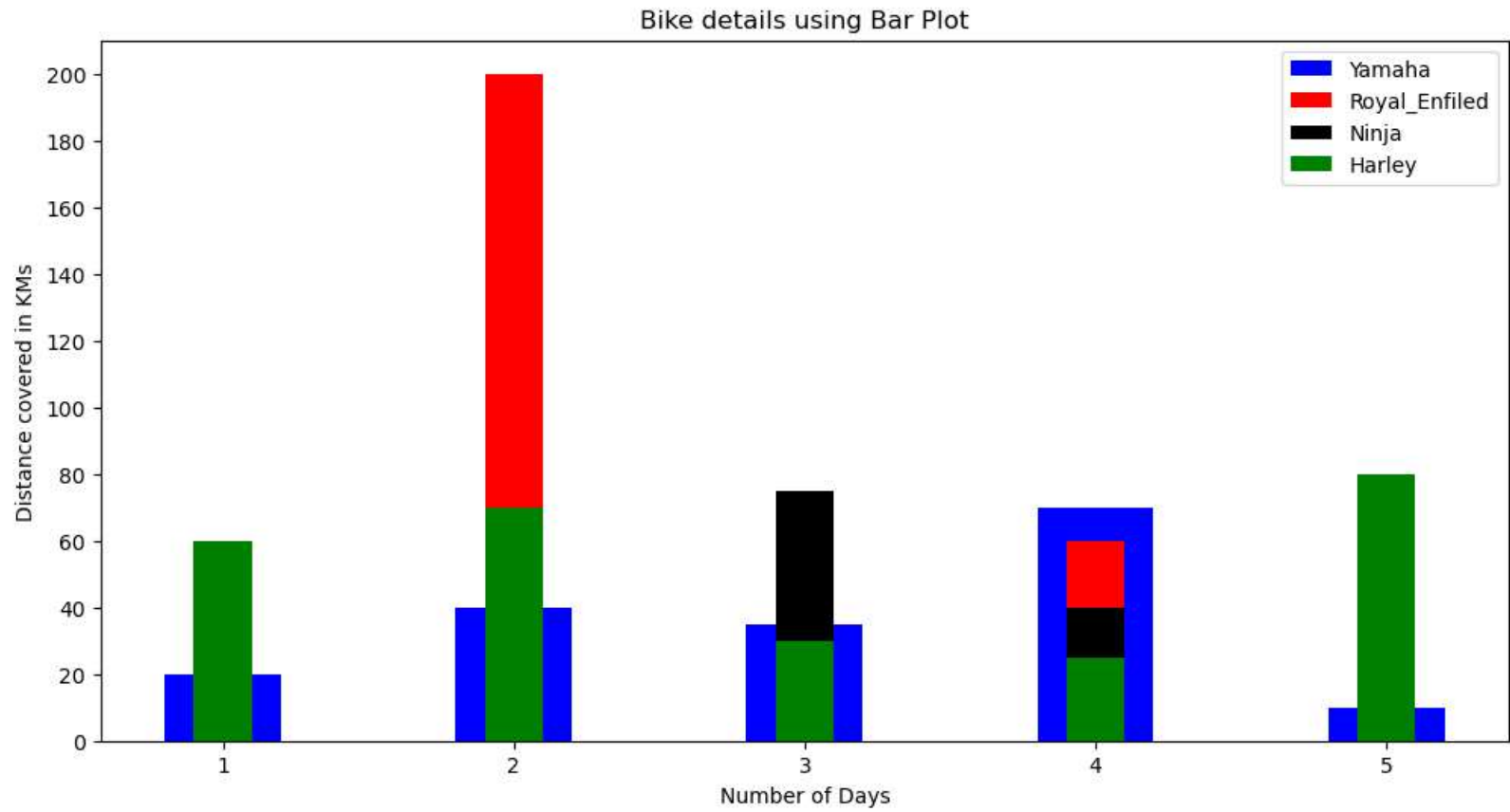
Bar Plot

```
In [13]: ▶ x = [1, 2, 3, 4, 5]

y1 = [20, 40, 35, 70, 10]
y2 = [10, 200, 50, 60, 30]
y3 = [30, 50, 75, 40, 60]
y4 = [60, 70, 30, 25, 80]

plt.figure(figsize=(12, 6))
plt.bar(x, y1, color='b', label='Yamaha', width=0.4)
plt.bar(x, y2, color='r', label='Royal_Enfield', width=0.2)
plt.bar(x, y3, color='k', label='Ninja', width=0.2)
plt.bar(x, y4, color='g', label='Harley', width=0.2)

plt.title('Bike details using Bar Plot')
plt.xlabel('Number of Days')
plt.ylabel('Distance covered in KMs')
plt.yticks(np.arange(0,201,20))
plt.legend()
# plt.grid()
plt.show()
```



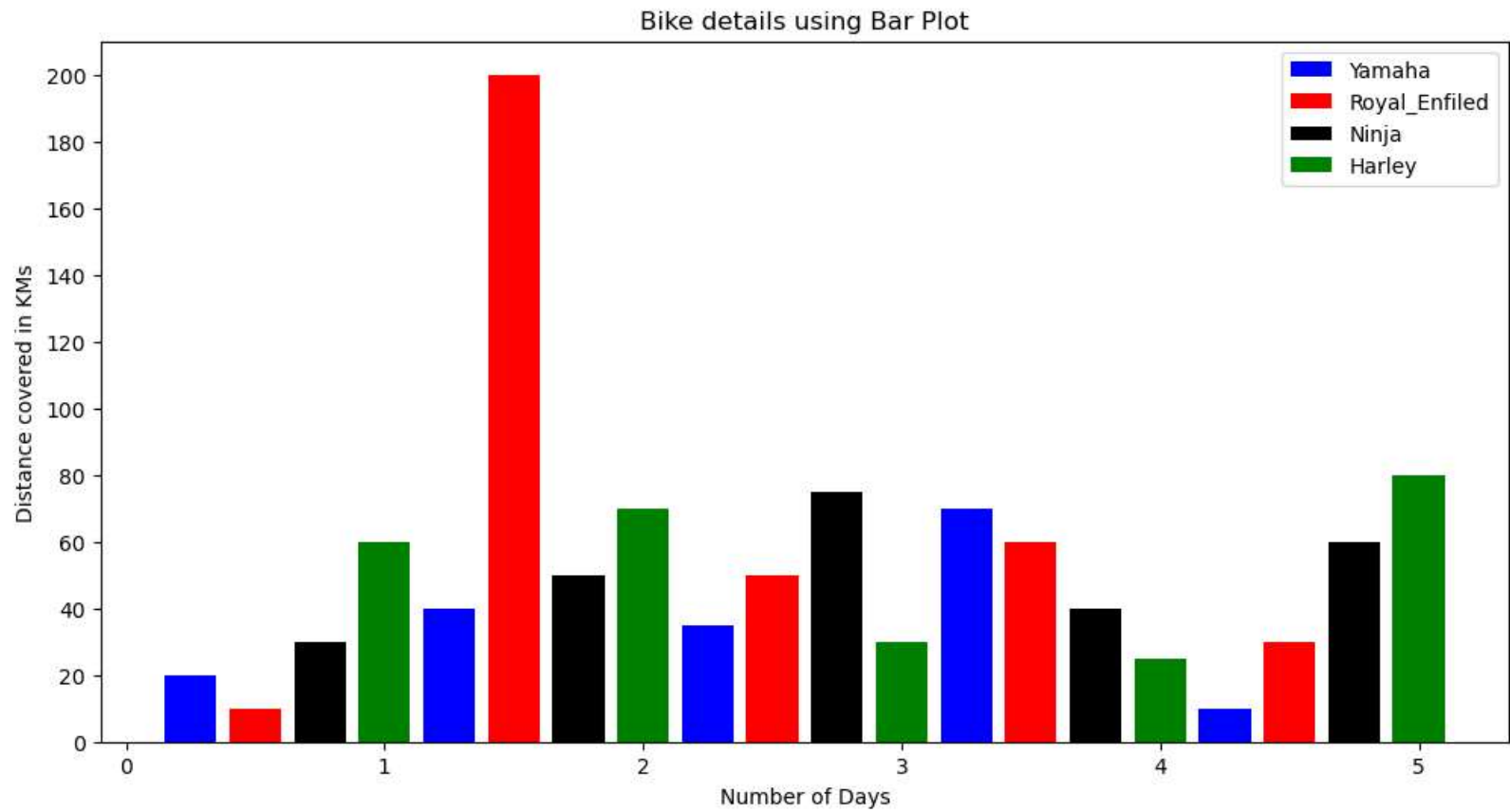
In []: ▶

```
In [15]: ▶ x1 = [0.25, 1.25, 2.25, 3.25, 4.25]
x2 = [0.5, 1.5, 2.5, 3.5, 4.5]
x3 = [0.75, 1.75, 2.75, 3.75, 4.75]
x4 = [1, 2, 3, 4, 5]

y1 = [20, 40, 35, 70, 10]
y2 = [10, 200, 50, 60, 30]
y3 = [30, 50, 75, 40, 60]
y4 = [60, 70, 30, 25, 80]

plt.figure(figsize=(12, 6))
plt.bar(x1, y1, color='b', label='Yamaha', width=0.2)
plt.bar(x2, y2, color='r', label='Royal_Enfield', width=0.2)
plt.bar(x3, y3, color='k', label='Ninja', width=0.2)
plt.bar(x4, y4, color='g', label='Harley', width=0.2)

plt.title('Bike details using Bar Plot')
plt.xlabel('Number of Days')
plt.ylabel('Distance covered in KMs')
plt.yticks(np.arange(0,201,20))
plt.legend()
# plt.grid()
plt.show()
```

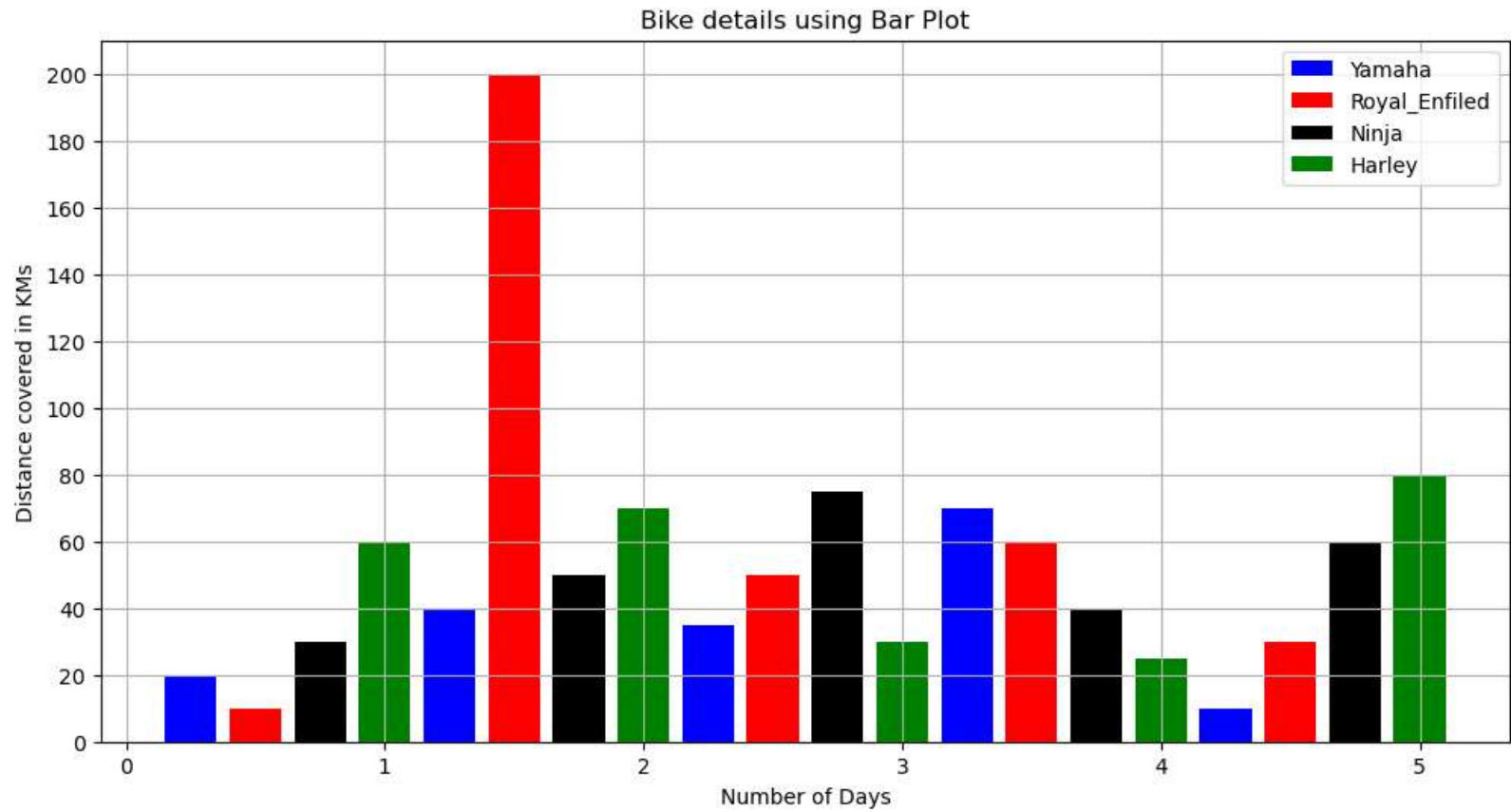



```
In [16]: ▶ x1 = [0.25, 1.25, 2.25, 3.25, 4.25]
x2 = [0.5, 1.5, 2.5, 3.5, 4.5]
x3 = [0.75, 1.75, 2.75, 3.75, 4.75]
x4 = [1, 2, 3, 4, 5]

y1 = [20, 40, 35, 70, 10]
y2 = [10, 200, 50, 60, 30]
y3 = [30, 50, 75, 40, 60]
y4 = [60, 70, 30, 25, 80]

plt.figure(figsize=(12, 6))
plt.bar(x1, y1, color='b', label='Yamaha', width=0.2)
plt.bar(x2, y2, color='r', label='Royal_Enfield', width=0.2)
plt.bar(x3, y3, color='k', label='Ninja', width=0.2)
plt.bar(x4, y4, color='g', label='Harley', width=0.2)

plt.title('Bike details using Bar Plot')
plt.xlabel('Number of Days')
plt.ylabel('Distance covered in KMs')
plt.yticks(np.arange(0,201,20))
plt.legend()
plt.grid()
plt.show()
```



In []: ▶

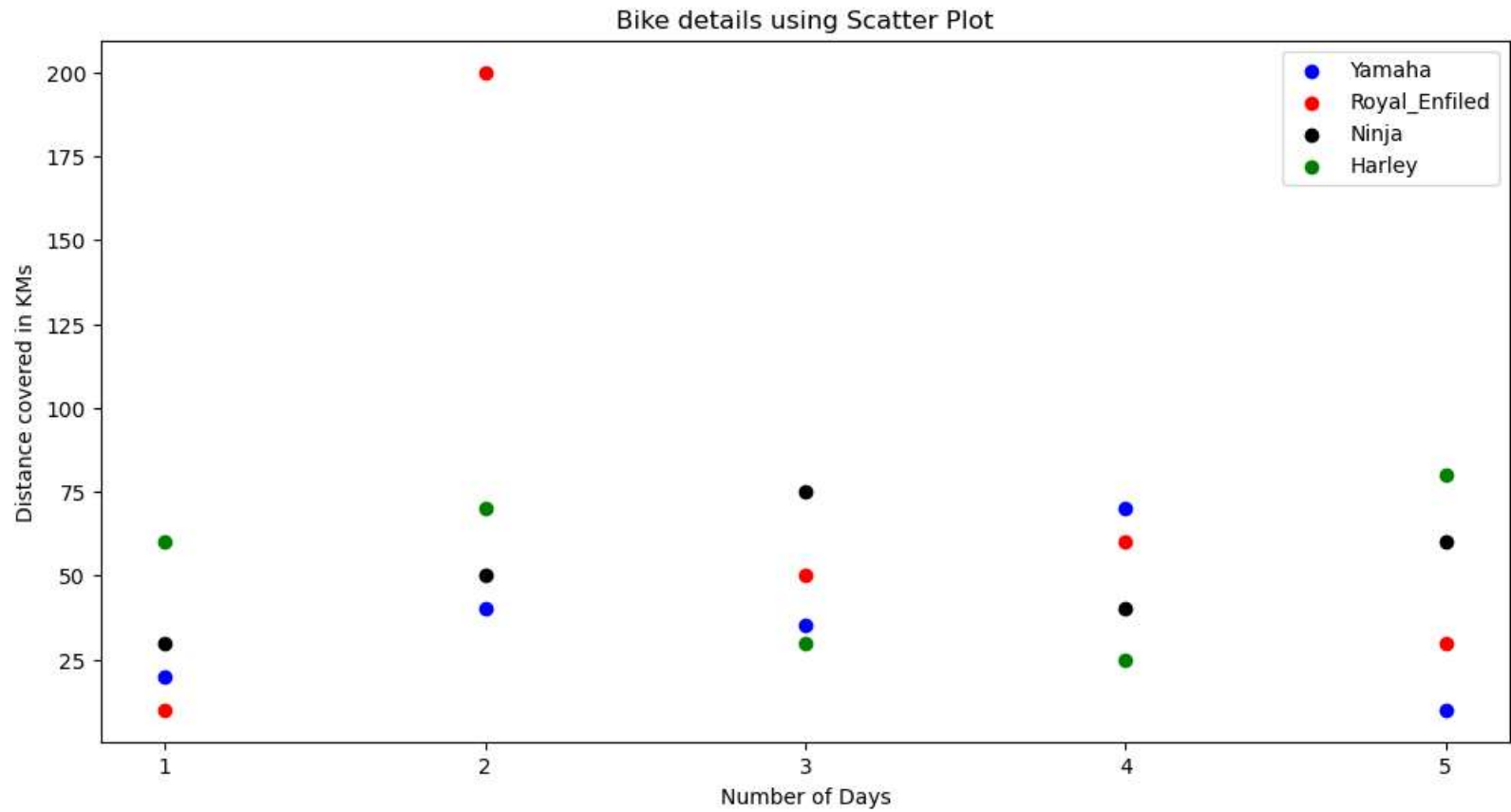
Scatter Plot

```
In [23]: ▶ x = [1, 2, 3, 4, 5]

y1 = [20, 40, 35, 70, 10]
y2 = [10, 200, 50, 60, 30]
y3 = [30, 50, 75, 40, 60]
y4 = [60, 70, 30, 25, 80]

plt.figure(figsize=(12, 6))
plt.scatter(x, y1, color='b', marker='o', label='Yamaha')
plt.scatter(x, y2, color='r', marker='o', label='Royal_Enfield')
plt.scatter(x, y3, color='k', marker='o', label='Ninja')
plt.scatter(x, y4, color='g', marker='o', label='Harley')

plt.title('Bike details using Scatter Plot')
plt.xlabel('Number of Days')
plt.ylabel('Distance covered in KMs')
plt.xticks(np.arange(1,6,1))
# plt.yticks(np.arange(10,201,10))
plt.legend()
# plt.grid()
plt.show()
```

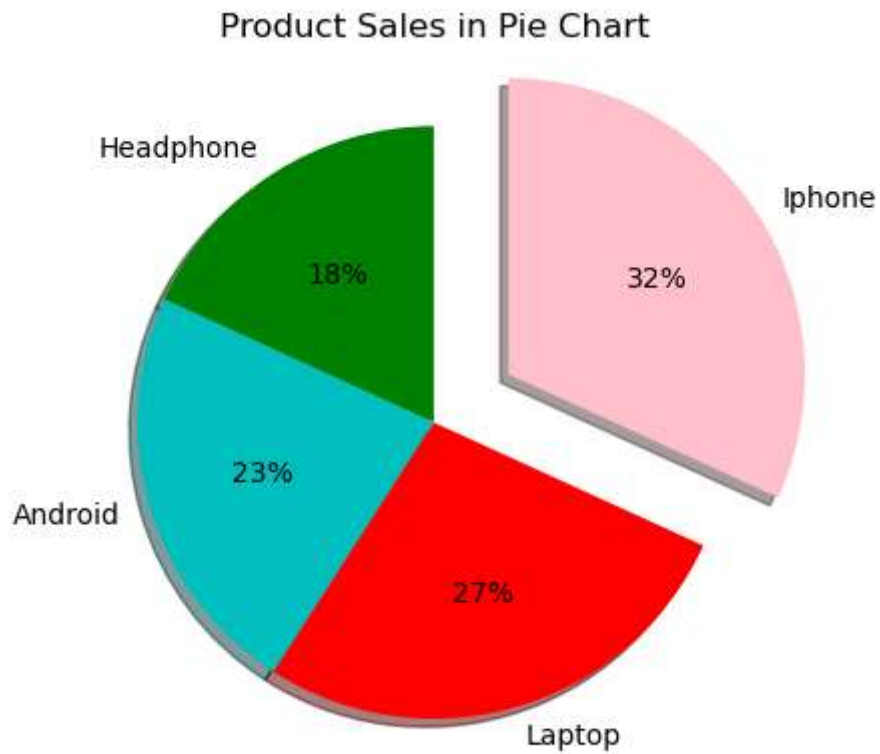


In []: ▶

Pie Plot

```
In [49]: ▶ slices = [4,5,6,7]
labels = ['Headphone', 'Android', 'Laptop', 'Iphone']
cols = ['g', 'c', 'r', 'pink']

plt.pie(slices, labels=labels, colors=cols, autopct='%1.0f%%', startangle=90, shadow=True, explode=(0,0,0,0))
plt.title('Product Sales in Pie Chart')
plt.show()
```



```
In [ ]: ▶
```

3D Graph

In []: ▶

```
In [54]: ▶ fig = plt.figure(figsize=(12, 6))
ax = fig.add_subplot(projection='3d')

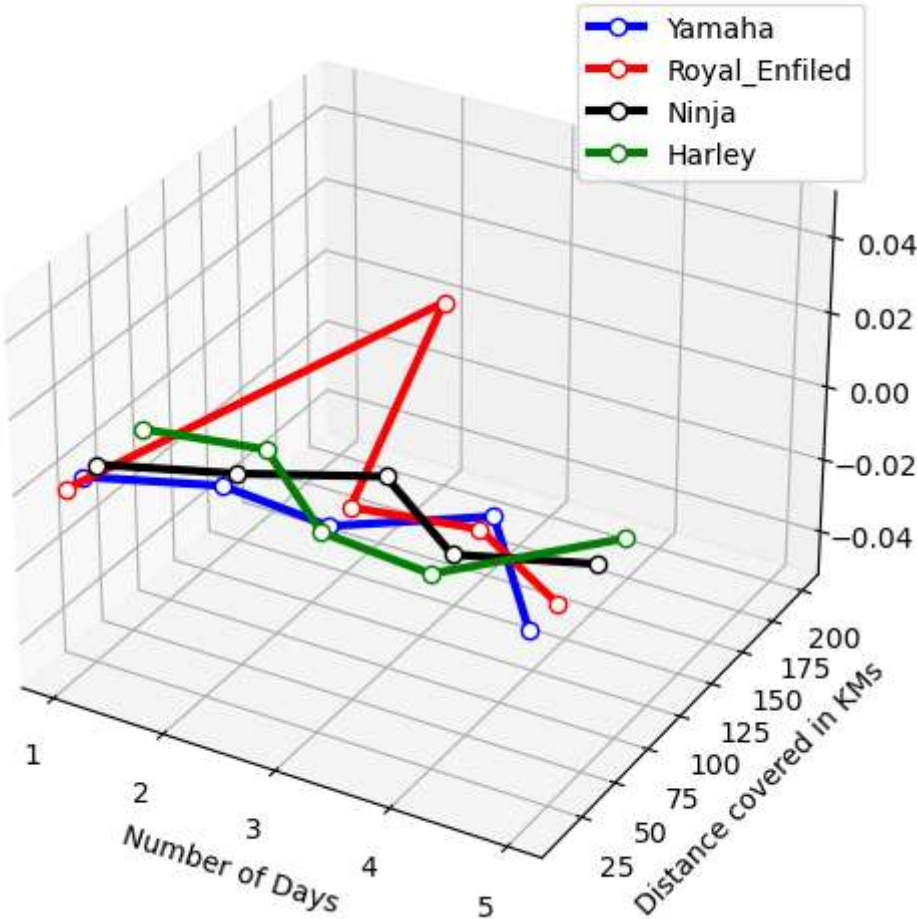
x = [1, 2, 3, 4, 5]

y1 = [20, 40, 35, 70, 10]
y2 = [10, 200, 50, 60, 30]
y3 = [30, 50, 75, 40, 60]
y4 = [60, 70, 30, 25, 80]

plt.plot(x, y1, color='b', marker='o', label='Yamaha', lw=3, markerfacecolor='white')
plt.plot(x, y2, color='r', marker='o', label='Royal_Enfield', lw=3, mfc='w')
plt.plot(x, y3, color='k', marker='o', label='Ninja', lw=3, mfc='w')
plt.plot(x, y4, color='g', marker='o', label='Harley', lw=3, mfc='w')

plt.title('Bike details using Line Plot')
plt.xlabel('Number of Days')
plt.ylabel('Distance covered in KMs')
plt.xticks(np.arange(1,6,1))
# plt.yticks(np.arange(10,201,10))
plt.legend()
# plt.grid()
plt.show()
```


Bike details using Line Plot



```
In [ ]: 
```

```
In [ ]: 
```

```
In [ ]: 
```

In []: ▶

Seaborn Library

```
In [61]: ▶ import seaborn as sns  
import pandas as pd
```

```
In [60]: ▶ sns.get_dataset_names()
```

```
anscombe,  
'attention',  
'brain_networks',  
'car_crashes',  
'diamonds',  
'dots',  
'dowjones',  
'exercise',  
'flights',  
'fmri',  
'geyser',  
'glue',  
'healthexp',  
'iris',  
'mpg',  
'penguins',  
'planets',  
'seaice',  
'taxis',  
'tips',  
....: .
```

In []: ▶

```
In [63]: ▶ df = sns.load_dataset('iris')
df
```

Out[63]:

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	virginica
146	6.3	2.5	5.0	1.9	virginica
147	6.5	3.0	5.2	2.0	virginica
148	6.2	3.4	5.4	2.3	virginica
149	5.9	3.0	5.1	1.8	virginica

150 rows × 5 columns

```
In [65]: ▶ df['species'].value_counts()
```

Out[65]: setosa 50
versicolor 50
virginica 50
Name: species, dtype: int64

```
In [68]: ▶ import warnings
warnings.filterwarnings('ignore')
```

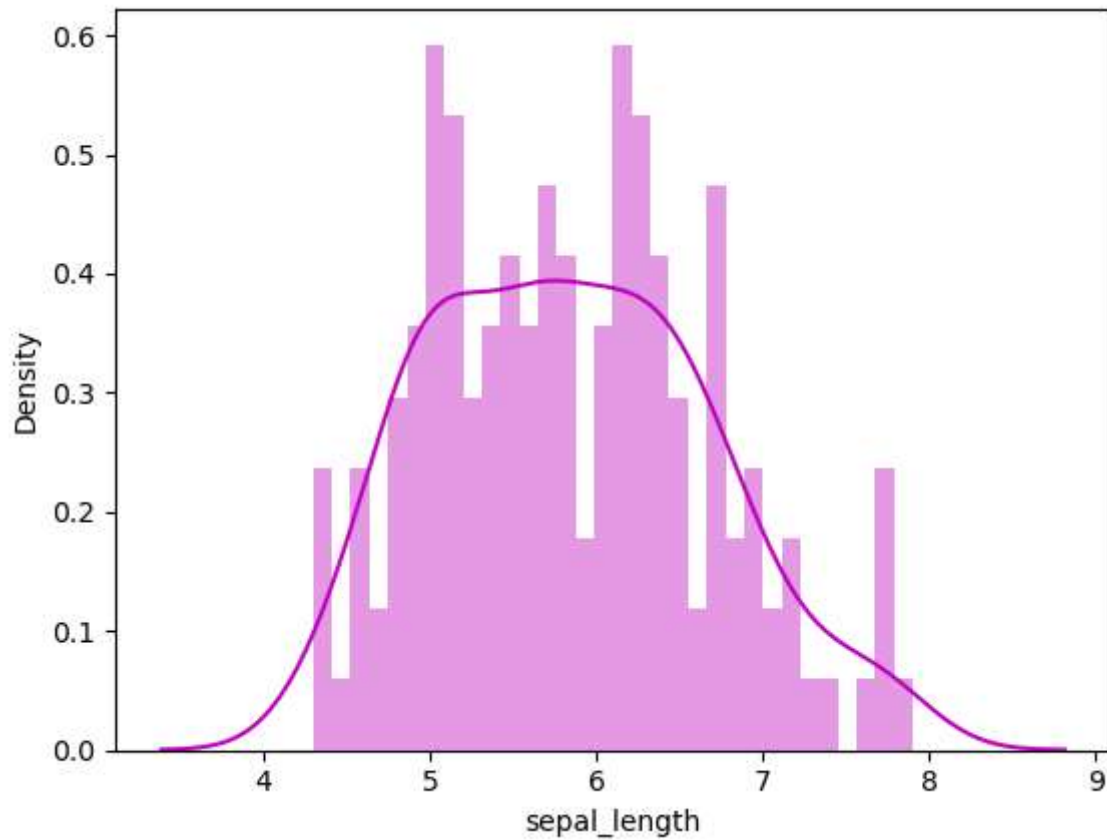
```
In [69]: df['sepal_length']
```

```
Out[69]: 0      5.1  
         1      4.9  
         2      4.7  
         3      4.6  
         4      5.0  
         ...  
        145     6.7  
        146     6.3  
        147     6.5  
        148     6.2  
        149     5.9  
        Name: sepal_length, Length: 150, dtype: float64
```

Distplot

```
In [82]: ▶ sns.distplot(df['sepal_length'], bins=32, kde=True, color='m')
```

```
Out[82]: <Axes: xlabel='sepal_length', ylabel='Density'>
```

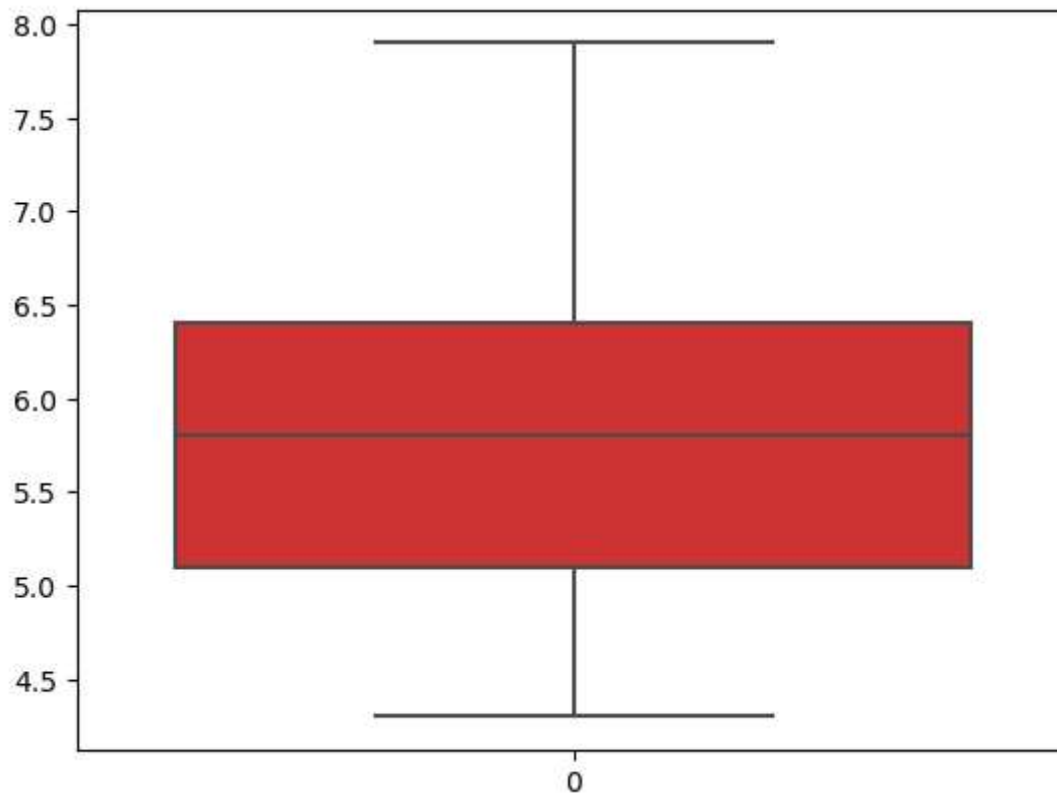


```
In [ ]: ▶
```

Boxplot

```
In [78]: ▶ sns.boxplot(df['sepal_length'], color='r', saturation=0.60)
```

Out[78]: <Axes: >



```
In [ ]: ▶
```

```
In [72]: df.describe()
```

Out[72]:

	sepal_length	sepal_width	petal_length	petal_width
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333
std	0.828066	0.435866	1.765298	0.762238
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

```
In [ ]:
```

```
In [83]: df.corr()
```

Out[83]:

	sepal_length	sepal_width	petal_length	petal_width
sepal_length	1.000000	-0.117570	0.871754	0.817941
sepal_width	-0.117570	1.000000	-0.428440	-0.366126
petal_length	0.871754	-0.428440	1.000000	0.962865
petal_width	0.817941	-0.366126	0.962865	1.000000

Heatmap

```
In [86]: ▶ plt.figure(figsize=(10,4))  
sns.heatmap(df.corr(), annot=True)
```

Out[86]: <Axes: >

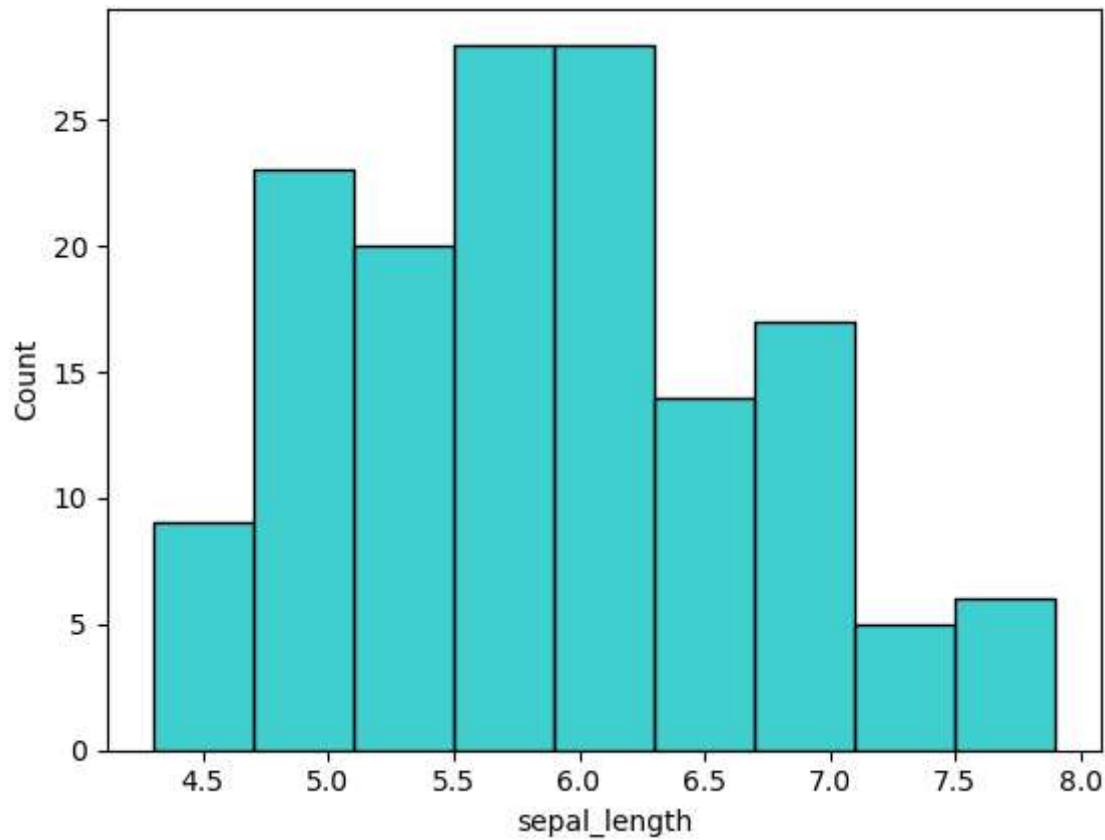


In []: ▶

Histogram


```
In [90]: ▶ sns.histplot(df['sepal_length'], color='c')
```

```
Out[90]: <Axes: xlabel='sepal_length', ylabel='Count'>
```

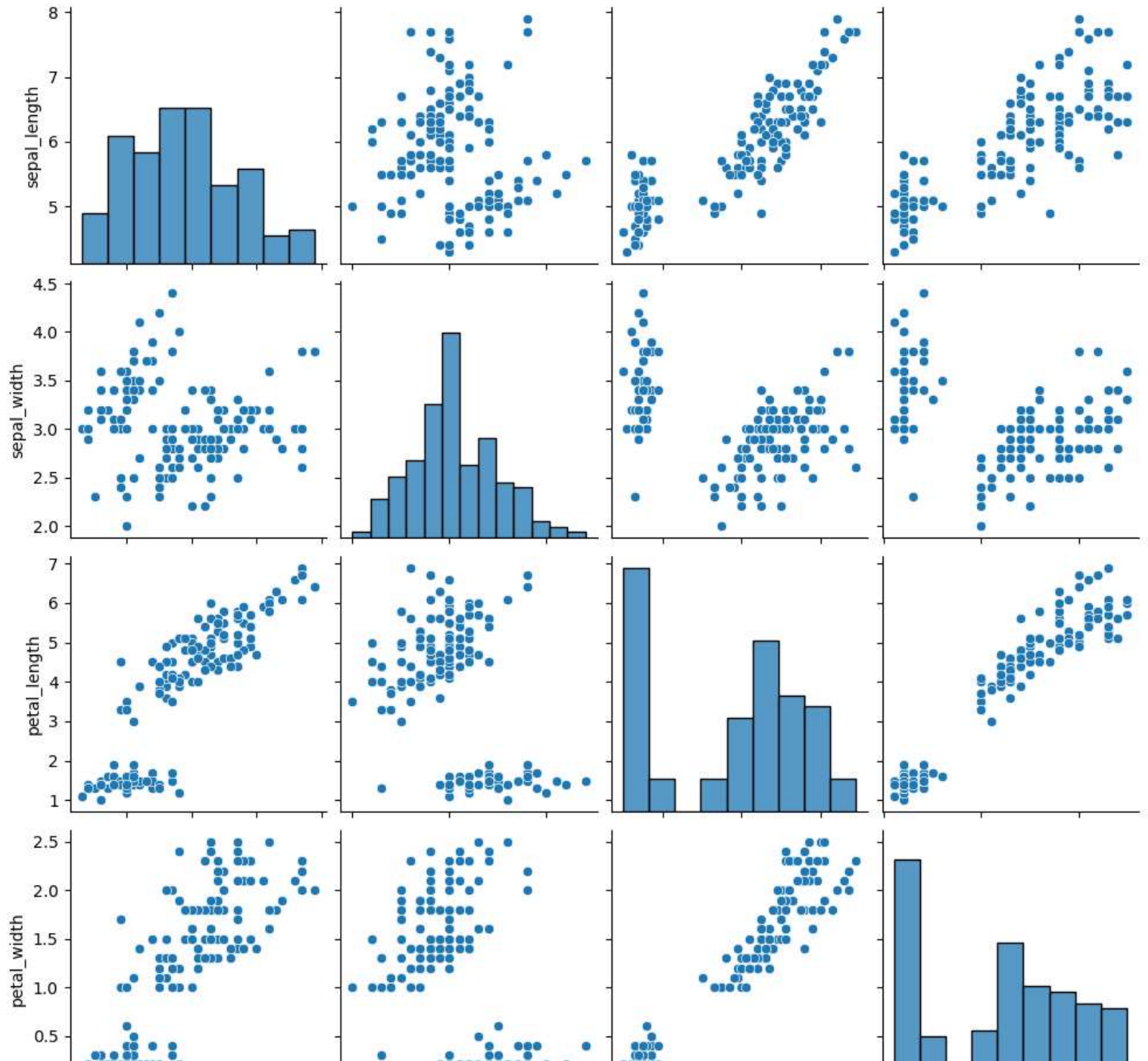


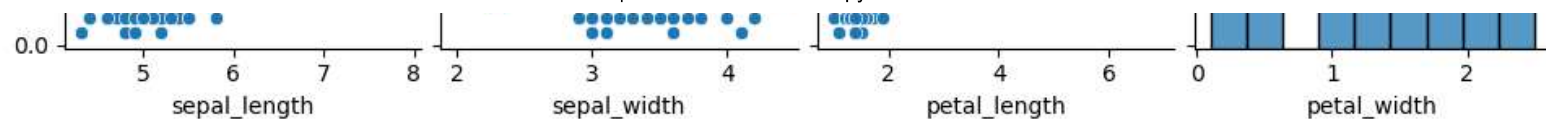
```
In [ ]: ▶
```

Univariate Analysis

```
In [93]: ▶ sns.pairplot(df)
```

```
Out[93]: <seaborn.axisgrid.PairGrid at 0x21bcdd40110>
```



In []: ▶

In []: ▶

In []: ▶

In []: ▶

In []: ▶

In []: ▶

In []: ▶

In []: ▶

In []: ▶